

Uart_Esp32

Projektdokumentation - Firmware

UART Monitor, Einstellungen und Decoder-Dauerbetrieb | Version v0.8

Projektname	Uart_Esp32
Dokumentenart	Projektdokumentation
Thema	Firmware fuer ESP32-WROOM-32 / NodeMCU-32S - UART Monitor, getrennte Einstellungen und Decoder-Dauerbetrieb
Erstellungsdatum	2026-09-17
Autor / verantwortliche Person	OpenAI ChatGPT (technische Erstellung)
Version	v0.8
Status	Entwurf / lokale Build- und Hardwarepruefung ausstehend
Aenderungsdatum	2026-09-17
Ablageort	Projektordner Uart_ESP32/docs
Referenzen	Regeln_Projektdokumentation_PDF_DOCX_Pflicht _(5).pdf; ESP32_UART_Protokoll_Entschluesselung(1).pdf; Projektstand v0.7

1. Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.1	2026-09-17	OpenAI ChatGPT	Entwurf	Reduzierter WROOM32-Projektstand erstellt
v0.2	2026-09-17	OpenAI ChatGPT	Entwurf	Projektname und Dokumentationsregeln umgesetzt
v0.3	2026-09-17	OpenAI ChatGPT	Entwurf	Typensichere Buildkorrekturen
v0.4	2026-09-17	OpenAI ChatGPT	Entwurf	Arduino-Secrets, NVS und WLAN-Provisionierung
v0.5	2026-09-17	OpenAI ChatGPT	Entwurf	WLAN-Fallbacks, Recovery-AP und Resetdiagnose
v0.6	2026-09-17	OpenAI ChatGPT	Entwurf	Compilerfehler in markWifiConnected behoben
v0.7	2026-09-17	OpenAI ChatGPT	Entwurf	UART Raw/Sniffer, Decoder, Feld-Aliase und Kalibrierung
v0.8	2026-09-17	OpenAI ChatGPT	Entwurf	UART-Monitor und Einstellungen

				getrennt; Raw/Decoder als persistenter Dauerbetrieb
--	--	--	--	--

2. Ziel des Standes v0.8

Der Projektstand v0.8 trennt die UART-Bedienung in eine reine Live-Monitorseite und eine explizite Einstellungsseite. Raw-/Sniffer-Modus und Protokoll-Decoder laufen nach Aktivierung dauerhaft in der ESP32-Hauptschleife weiter, auch wenn keine Browserseite geoeffnet ist. Der gewaehlte Modus wird in NVS gespeichert und nach einem normalen Neustart automatisch wieder gestartet.

Die technische Kennung Feld 1 bis Feld 6 bleibt immer sichtbar. Zu jedem Feld kann ein frei editierbarer Alias gespeichert werden, z. B. Feld 1 - Roll. Dadurch koennen spaeter bestaetigte Bedeutungen eingetragen werden, ohne die neutrale Feldnummerierung zu verlieren.

3. UART-Betriebsarten und Hardwarekonfiguration

Modus	Verhalten
Aus	Zusaetzliche Hardware-UART ist deaktiviert.
Raw / Sniffer	Empfangene Bytes werden dauerhaft in einem 512-Byte-Ringpuffer erfasst; die letzten 256 Bytes werden als HEX und ASCII dargestellt. Bei vollem Puffer werden nur die aeltesten Bytes ueberschrieben.
Protokoll-Decoder	Raw-Erfassung bleibt dauerhaft aktiv; zusaetlich werden Pakete kontinuierlich synchronisiert, geprueft und dekodiert. Der Browser muss dafuer nicht geoeffnet sein.

UART0 bleibt fuer den seriellen Debug-Monitor reserviert. Fuer die Zusatzfunktion sind Hardware-UART1 und UART2 waehlbar. RX und TX sind frei konfigurierbar; TX kann mit -1 deaktiviert werden. Standard ist RX GPIO16, TX deaktiviert, 115200 Baud und 8N1.

Dauerbetrieb: Die UART-Verarbeitung wird kontinuierlich in loop() aufgerufen und besitzt keine Messanzahl oder Laufzeitbegrenzung. Der 512-Byte-Ringpuffer kann nicht volllaufen, weil bei Bedarf nur die aeltesten Rohbytes ueberschrieben werden. Die Dekodierung selbst laeuft weiter und ist nicht an eine geoeffnete Browserseite gebunden.

- GPIO6..11 sind wegen des externen Flashs gesperrt.
- GPIO1/3 bleiben fuer den Debug-UART reserviert.
- GPIO34..39 duerfen nur als RX verwendet werden, da sie reine Eingänge sind.
- Unterstuetzte Frame-Formate: 8N1, 8E1, 8O1 und 8N2.

4. Decoder gemaess UART-Messunterlage

Der Decoder folgt der im Referenz-PDF dokumentierten Paketlogik. Die robustere Synchronisation interpretiert nicht einfach jedes 25-Byte-Fenster, sondern sucht zuerst den Paketheader und verwendet das Laengenbyte.

1. Suche nach FF.

2. Folgendes Byte muss FB sein.
3. Typ und Laengenbyte lesen.
4. Gesamtlänge als 4 + Länge bestimmen.
5. Vollständiges Paket sammeln.
6. XOR-Prüfsumme über Byte 6 bis zum Byte vor der Prüfsumme berechnen.
7. Nur gültige Pakete auswerten.

Für das bestätigte Hauptpaket Typ 0x10 und Länge 0x15 ergibt sich eine Gesamtlänge von 25 Bytes. Die sechs Nutzfelder beginnen bei Byte 12 und sind 16-Bit Little Endian.

Verifikationshinweis: Die Implementierung folgt der im PDF textlich angegebenen XOR-Regel (Byte 6 bis zum Byte vor der Prüfsumme). Die im PDF abgedruckte einzelne Beispiel-Bytefolge ergibt bei einer unabhängigen Nachrechnung nicht denselben Endwert wie das dort gezeigte letzte Byte. Deshalb zeigt die Firmware gültige/ungültige Paketzähler separat an; der Hardwaretest muss bestätigen, ob die XOR-Grenzen für die realen Mitschnitte exakt stimmen. Der Raw-Modus bleibt hiervon unberührt.

Technisches Feld	Byteposition	Behandlung
Feld 1	12 / 13	uint16 LE, normiert
Feld 2	14 / 15	uint16 LE, normiert
Feld 3	16 / 17	uint16 LE, normiert
Feld 4	18 / 19	uint16 LE, normiert
Feld 5	20 / 21	uint16 LE, normiert
Feld 6	22 / 23	uint16 LE, nur Rohwert
Checksumme	24	XOR

5. Normierung, Totzone und Kalibrierung

Feld 1 bis Feld 5 werden stückweise linear auf -1.0 ... +1.0 normiert. Minimum, Mittelpunkt und Maximum werden pro Feld getrennt gespeichert. Als Startwerte werden die Werte aus der Referenzmessung verwendet.

Feld	Minimum	Mitte	Maximum
Feld 1	1000	1503	2000
Feld 2	1000	1486	2000
Feld 3	1000	1493	2000
Feld 4	1006	1511	2000
Feld 5	1000	1498	2000

Nach der Normierung kann eine Totzone angewendet werden. Standard ist 0.03. Feld 6 bleibt bewusst als Rohwert sichtbar, weil seine Bedeutung in der Messunterlage noch nicht abschliessend bestätigt ist.

6. Änderbare Feldnamen bei stabiler technischer Kennung

Für Feld 1 bis Feld 6 ist jeweils ein freies Alias-Textfeld mit maximal 32 Zeichen vorhanden. Der Alias ersetzt die technische Kennung nicht, sondern wird zusätzlich angezeigt.

Feld 1 - Roll
 Feld 2 - Pitch
 Feld 3 - noch unbekannt

Solange die Semantik eines Feldes nicht sicher bestaetigt ist, kann der Alias leer bleiben oder z. B. noch unbekannt enthalten. Die Aliase werden in NVS gespeichert und auch ueber MQTT bereitgestellt.

7. Webinterface

Ab v0.8 sind Konfiguration und Beobachtung getrennt. /uart ist eine reine Live-Monitorseite; /uart/settings enthaelt Betriebsart, Hardware-UART, GPIOs, Baudrate, Format, Feld-Aliase, Totzone und Kalibrierung. Die Liveanzeige wird einmal pro Sekunde ueber /uart/status aktualisiert. Das ist nur die Darstellung; Empfang und Dekodierung laufen davon unabhaengig im Hintergrund weiter.

- Monitor /uart: Live-Status, Feldwerte, Paketzaehler, Raw-HEX und ASCII
- Einstellungen /uart/settings: Modus Aus / Raw / Decoder
- Hardware-UART1 oder UART2 sowie RX- und optionaler TX-GPIO
- Baudrate und UART-Frame
- Alias-Textfelder fuer Feld 1 bis Feld 6
- Min/Mitte/Max fuer Feld 1 bis Feld 5 und Totzone
- Gespeicherter Raw-/Decoder-Modus startet nach ESP32-Neustart automatisch wieder
- Monitorseite kann geschlossen werden; Dekodierung laeuft weiter
- Paketzaehler und Checksumme-Statistik
- Raw-HEX- und ASCII-Anzeige
- Rohdatenpuffer / Bytezaehler loeschen, ohne den Decoder zu stoppen

Route	Methode	Funktion
/uart	GET	Reine UART-Liveansicht / Decoder-Monitor
/uart/status	GET	JSON-Livestatus fuer die Monitorseite
/save_uart	POST	Einstellungen in NVS speichern und UART neu initialisieren
/uart_clear	POST	Raw-Ringpuffer und Bytezaehler leeren; Decoderbetrieb bleibt aktiv
/uart/settings	GET	Explizite UART-Konfiguration, Feld-Aliase und Kalibrierung

8. MQTT-Erweiterung

Der normale Heartbeat enthaelt zusaetzlich den UART-Modus und den Bytezaehler. Unter <Basis>/uart/ werden Status, Paketzaehler und dekodierte Feldwerte veroeffentlicht.

- mode, status, bytes
- packets, valid_packets, invalid_packets
- field1/raw bis field6/raw
- field1/norm bis field5/norm
- field1/alias bis field6/alias (retained)

Der Raw-HEX-Datenstrom wird absichtlich nicht vollstaendig ueber MQTT gespiegelt, um Broker und Funkverbindung nicht mit ca. 50-Hz-Telegrammen zu belasten.

9. NVS-Persistenz

UART-Einstellungen liegen im separaten Namespace `uartconf`. Gespeichert werden Betriebsart, Hardware-UART, RX/TX, Baudrate, Frame, Totzone, sechs Aliase sowie die Kalibrierwerte fuer Feld 1 bis Feld 5. Fehlende Keys werden vor dem Lesen mit `isKey()` geprueft und erhalten sichere Defaults.

10. Elektrische Voraussetzungen

Der ESP32 arbeitet mit 3,3-V-Logik und seine GPIOs sind nicht 5-V-tolerant. Fuer erste Messungen soll nur RX angeschlossen werden.

- GND des ESP32 und GND des Zielgeraets verbinden.
- High-Pegel der UART-Leitung vor dem Anschluss messen.
- Bei 5 V Pegelwandler oder geeigneten Spannungsteiler verwenden.
- TX erst nach Klaerung von Richtung und Bus-Topologie verbinden.

11. Projektstruktur und geaenderte Dateien

```
Uart_ESP32/
.gitignore
platformio.ini
README.md
include/
  arduino_secrets.example.h
  arduino_secrets.h      # lokal; nicht im Auslieferungs-ZIP
src/
  ConfigDefaults.h
  BatteryMonitor.cpp/h
  DeepSleepManager.cpp/h
  MqttManager.cpp/h
  OtaManager.cpp/h
  UartManager.cpp/h      # neu v0.8
  main.cpp
docs/
  2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.8.*
  2026-09-17 - Uart_Esp32 - Commit-Nachricht - UART-Seitentrennung-und-Dauerbetrieb - v0.8.txt
```

Der Projektname bleibt `Uart_Esp32`. Der Root-Ordner `Uart_ESP32` wird fuer diese Auslieferung unveraendert aus dem vom Nutzer bereitgestellten Paket uebernommen, damit die Paketstruktur gemaess Projektdokumentationsregel erhalten bleibt.

Geaenderte / neue Dateien:

- README.md
- src/ConfigDefaults.h
- src/main.cpp
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.8.md/.docx/.pdf
- docs/2026-09-17 - Uart_Esp32 - Commit-Nachricht - UART-Seitentrennung-und-Dauerbetrieb - v0.8.txt
-
- docs/2026-09-17 - Uart_Esp32 - Projektdokumentation - Firmware - v0.8.md/.docx/.pdf

- docs/2026-09-17 - Uart_Esp32 - Commit-Nachricht - UART-Seitentrennung-und-Dauerbetrieb - v0.8.txt

12. Pruefung und offene Verifikation

Die Aenderungen wurden statisch auf Einbindung, NVS-Key-Laengen, Parsergrenzen, Pinvalidierung und Dokumentationsstruktur geprueft. In der Erstellungsumgebung steht kein PlatformIO-Compiler fuer das ESP32-Arduino-Framework zur Verfuegung. Deshalb ist der lokale Build weiterhin verbindlich.

```
pio run -e uart_esp32_usb -t clean
pio run -e uart_esp32_usb
pio run -e uart_esp32_usb -t upload
```

Der erste Hardwaretest sollte ueber /uart/settings im Raw-/Sniffer-Dauerbetrieb mit TX = -1 erfolgen. Auf /uart muss die Bytezahl kontinuierlich steigen. Erst nach bestaetigter 3,3-V-Pegellage und sichtbaren 115200-8N1-Daten sollte der Protokoll-Decoder-Dauerbetrieb aktiviert werden. Danach darf die Monitorseite geschlossen werden; Empfang und Dekodierung muessen weiterlaufen.

13. Auslieferungsform

Die Auslieferung enthaelt ein vollstaendiges ZIP mit unveraendertem Root-Ordner Uart_ESP32, ein ZIP mit nur geaenderten/neuen Dateien in derselben Struktur, eine Patch-Datei, eine englische BL_-Commit-Nachricht mit Datum und Dateiliste sowie Projektdokumentation als PDF und bearbeitbare DOCX mit identischem Stand v0.8. Die lokale arduino_secrets.h wird nicht ausgeliefert.